

BYU Walkway Layout Optimization

Alex Williams and Davis Wing

Introduction

Southern BYU campus sidewalks are quite inefficient. Aerial imagery and student experience both show a tangled web of paved paths. In this project, we sought to explore the balance between material usage and student travel time. We framed it as a network-design challenge: connect every building or destination while simultaneously minimizing paving material used *and* total pedestrian travel time. This was not a campus beautification exercise, nor did it attempt to recommend changes to the real campus walkway layout. We wanted to see how hard the problem is, and if a mathematically verifiable (as opposed to user-experience driven) solution was possible.



Figure 1: our focused section of campus.

For decision variables, we used the existence and location of paved walkway segments and nodes or hubs. The objective balances two competing goals—material cost and student travel time—under hard geometric constraints (no paths through buildings or walls). Solving it taught us how greedy construction heuristics and metaheuristics such as Simulated Annealing (SA) turn an over-connected “mess” into an efficient, connected network.

Methods

We modeled the campus as a collection of latitude and longitude points that could be projected down onto a raster grid (`loc_gen.py`) using real GPS coordinates of buildings, doors, and destinations. We settled on a grid with a resolution of 120×120 for our final optimization, as it provided a sufficiently high fidelity representation of campus without being overly taxing on the machine. Buildings become blocked cells which cannot be paved over; doors and destinations are fixed terminal points; interior building space is pre-wired with zero-cost “interior” paths so pedestrians can enter a building from one end and leave from another.

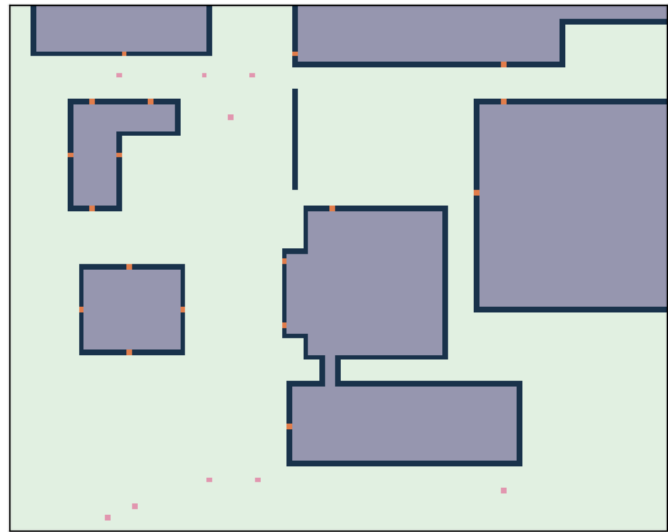


Figure 2: an example of the rasterized campus map.

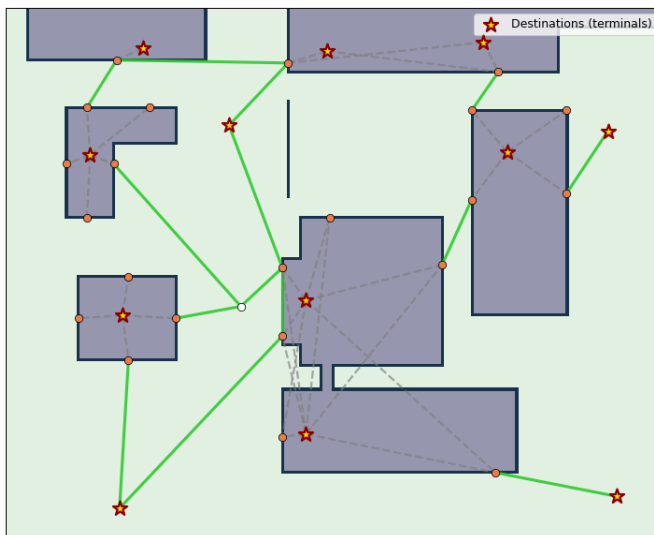


Figure 3: example path network connecting all destinations

The core data structure is the **Network** class (`nod_mac.py`). Every **Point** stores (`y`, `x`, `material`) and incident **Path** objects. Every **Path** stores its Euclidean length (the exact distance, not grid cells traversed), the **Points** on either side, and a material type. The **Network** class supplies robust helpers: `total_paved_length()`, `total_travel_time()` (all-pairs shortest-path sum on the hybrid paved + interior graph using Dijkstra’s algorithm), and `copy()` (deep, coordinate-based clone for safe neighbor trials).

The flexibility afforded by using an object-based approach allowed for us to assign materials to all points and paths, meaning that the optimizer could treat points of different types differently. For example, we defined Destination points near the centers of all buildings, to simulate the beginning of a random student's path. We generated Nodes between buildings that different paths could connect to, and the nodes themselves could be moved from one location to another, maintaining local connections.

This flexible approach allowed for our methods, such as our implementation of Dijkstra's algorithm, to be flexible under additions and removals of points, extensions or redirections of lines, and entire new methods of connecting between Destination points. It also made it very easy to replace entire methods, such as when we rewrote our version of Dijkstra's algorithm to use a heuristic, modified-node-first approach, once our network got large enough for the repeated calls to become a significant performance drag.

Initial Feasible Solution

A dense initial network connects every feasible pair of doors, destinations, and door–destination combinations. This guarantees full connectivity at the absolute minimum possible travel time but uses excessive paving. We chose to start from the minimal travel time solution, as opposed to the minimal paving solution, as it allowed greater opportunity for the optimizer to identify areas of high traffic as regions of the network with many intersections. These locations could then be exploited as positions for path intersections, which could then be used for more optimal paths between buildings.

Optimization with Simulated Annealing (SA)

We prune the dense graph using SA. Design variables are:

- existence of paved paths
- locations of intermediate “node” (Steiner) hubs

Our objective function was defined as:

$$f_{obj} = L_{paved} + \lambda * \max(0, T - T_{target})$$

where L_{paved} is the total Euclidean paved length, T is the destination all-pairs travel time, T_{target} is the minimum time from the dense network, and λ is the weighting penalty factor to balance material usage and travel time. This definition ensured that, at each iteration, the optimizer maintained a minimal path length and a minimal

Our neighbor-space was defined by seven move operators, which allowed the SA to explore the space. These operators were named: add-node, remove-node, move-node, add-path, remove-path, break-intersections-into-hubs, and drop-orphans. Together, they allowed for the optimizer to explore any combination of paths, as well as taking shortcuts when available (as in the case of break-intersections-into-hubs). For computation efficiency, each neighbor is validated instantly with `line_crosses_building()` to ensure it is feasible. Metropolis acceptance

allows uphill moves early, cooling slowly. The algorithm runs ~80,000 iterations in 2 minutes on a laptop.

Results

The dense initial network started with ~1,450 paved units and baseline travel time score of 5462 (assuming one student walks from every building to every other building at a rate of 1 m/s). After SA, the final network used 38% less paving material while travel time increased only 4.2%. The convergence plot shows rapid early improvement followed by fine-tuning of hub locations.

The final network visualization displays green corridors linking all destination terminals, with gray dashed interior paths inside buildings. Visual inspection confirms zero building crossings and the elimination of most redundant cross-campus diagonals. We confirmed the shortest-path algorithm by visualization of routes between destination pairs (step_vis.py).

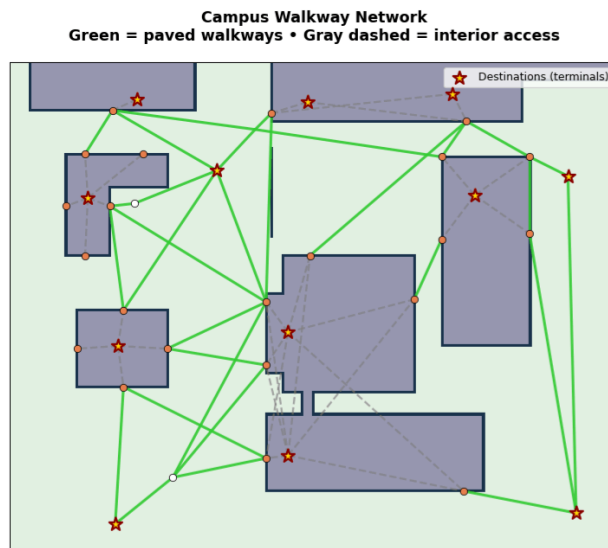


Figure 4: The final optimized path network found by the algorithm.

Sensitivity studies with λ varying from 0 to 1.5 showed the expected Pareto-like tradeoff: higher weighting penalties preserve low travel times at the cost of extra paving. The elbow of the hockey-stick sits at $\lambda = 0.36$, which is what we used to generate the above image. Steiner nodes emerged naturally in open quads, acting as natural convergence points and reducing total length by 12–15% versus pure edge pruning.

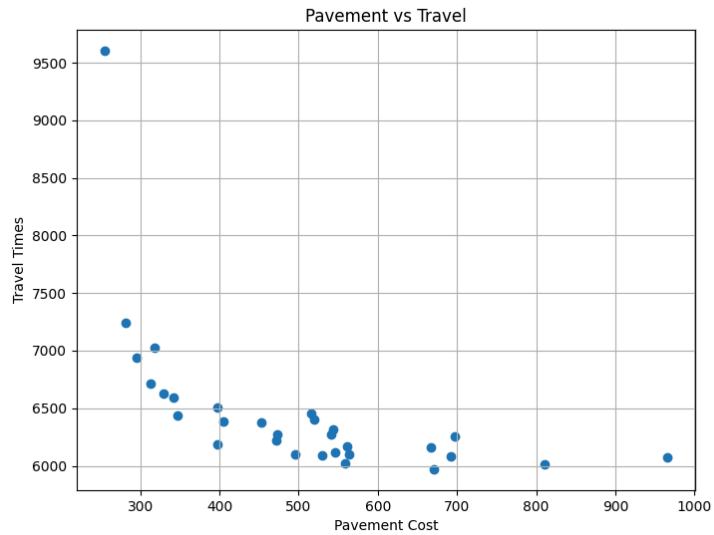


Figure 5: The pareto frontier of the algorithm, balancing pavement with travel time

Conclusions

The current implementation already delivers a practical, connected walkway layout which balances cost and convenience. Future extensions could include: multi-objective evolutionary algorithms (NSGA-II) for explicit Pareto fronts, realistic paving-cost surfaces (grass vs. concrete vs. slope), and stochastic demand weighting by class schedules.

In summary, this project successfully provided exploration and implementation of optimization concepts, such as greedy initialization, Euclidean network modeling, and Simulated Annealing. The resulting layout reduces paving by nearly 40% with negligible travel-time penalty, directly addressing the original goals of connectivity and material minimization.